DELPHI WEB STUDIO PRO

Engineering Guide

How the Windows host editor model and website exporter work together

This guide is for maintaining and extending the application. It explains who owns each kind of state, how operations cross the Windows boundary, and why an editing change may also need work in persistence or export. Examples trace the implementation rather than teaching the reader where to click.

Windows desktop edition
Last changed November 10, 2026

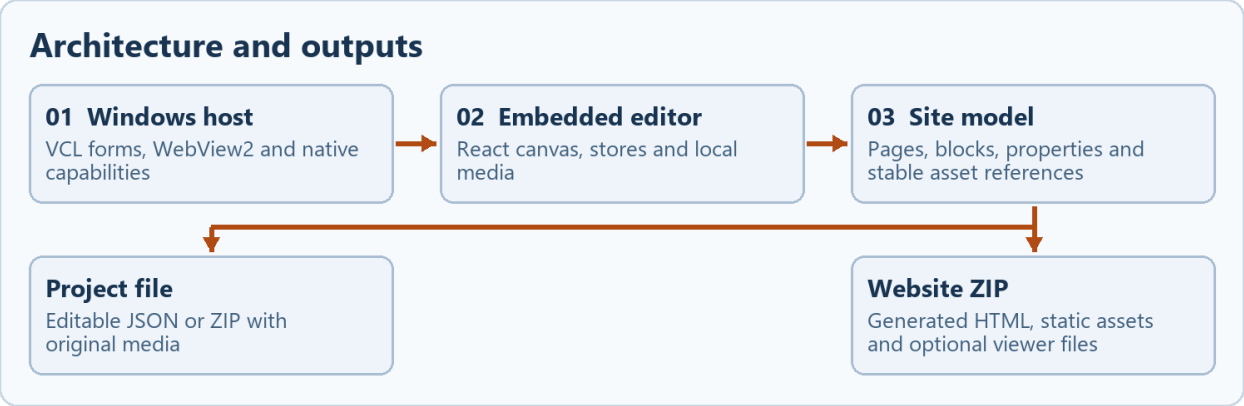
Application source reviewed at revision 61b52c9.

Contents

The application has three connected products....3
Startup begins in VCL and hands display to WebView2....4
The native bridge grants narrow capabilities....5
State ownership explains what can change independently....5
The portable project model preserves editable structure....6
Store actions connect edits history and shared sections....7
Selection connects the canvas to focused properties....7
Geometry panels and collages use different local spaces....8
Rich text combines editable DOM with persistent runs....9
Responsive layout and page navigation need parity....9
Media identity is independent of folder organization....10
PDFs and visitor components keep local interaction state....10
Opening compares snapshots before activating a project....11
Saving confirms a snapshot before clearing dirty state....12
Native replacement and recovery protect different copies....12
Import and export transform the model at different boundaries....13
Deployment and AI remain explicitly reviewed operations....14
Build a coherent package and test the affected boundary....15

The application has three connected products

Delphi Web Studio Pro combines an IDE-editable VCL host with a React and TypeScript editor running inside WebView2. The host provides Windows integration and controls privileged operations. The editor owns the site model and the visual authoring experience. A separate exporter turns that model into HTML, styles, scripts and assets for visitors, who do not need the desktop application.



The Windows host supplies native services around an editor whose model also feeds the independent website exporter.

Area	Implementation
Editor bundle	Vite builds index.html and hashed assets. WebView2 loads this bundle as the application UI.
Portable project	The editor serializes its site model and, when media is present, packages original media in a ZIP. This file can be reopened to edit the site.
Website output	The exporter generates HTML and supporting assets for visitors. It consumes the project model but has its own rendering and path rules.

These boundaries help locate a change. A native menu or dialog belongs to the Pascal and DFM pair. Selection or image geometry belongs to the editor. Generated URLs and HTML belong to the exporter. A visitor-facing layout change often spans both rendering paths, because an editable React component is not automatically the static HTML implementation.

Area	Implementation
desktop/DelphiWebStudio	IDE-editable .dfm forms, their .pas event handlers, the .dpr entry point, .dproj build settings and resources.
desktop/bridge	Native protocol validation, capability sessions and Windows credential functions.
desktop/project	Native file open, save, backup, restore, SFTP, deployment review, journals and rollback.
desktop/ai	Staging and startup of the local runtime, process ownership and the local HTTP client.
src/routes and src/editor	App routes, editor shell, canvas, toolbar, palette, layer list, property windows and code panels.

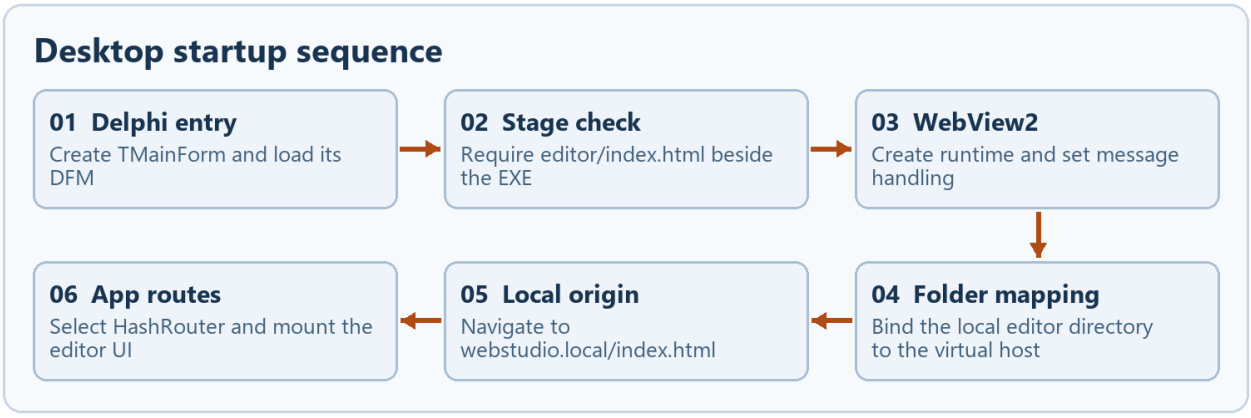
Area	Implementation
src/blocks and src/store	Persisted block types, React renderers and layout wrappers, project and configuration stores, session state.
src/project and src/media	Versioned project parser, portable file commands, importers, checks, asset storage, PDF preview and picker.
src/lib and src/ai	Theme presets, starter templates, content helpers, website renderer, site manifest and AI action validation.
api and tests	Optional server endpoints, browser and unit test suites, native Delphi harnesses and loopback deployment tests.
docs and desktop/installer	Documentation, license material, handoff and Inno Setup packaging inputs.

The application source reviewed for this edition is revision 61b52c9. Earlier milestone records remain useful evidence of particular tests, but their dates and stage names should be read as history rather than instructions for the current package.

Startup begins in VCL and hands display to WebView2

DelphiWebStudioPro.dpr initializes VCL, enables the main form on the taskbar, creates TMainForm and enters the application message loop. WebStudio.Main.dfm supplies the designer-backed controls, event wiring and maximized state. The paired Pascal unit implements their behavior, including the creation of session dictionaries that associate opened projects with disk paths and file identities.

FormCreate checks for editor\index.html beside the running executable before starting WebView2. The completion handler verifies the required WebView2 interface, configures settings and maps the adjacent editor directory to https://webstudio.local. The mapped origin supplies local application assets without requiring a Vite server. Failure updates the native status instead of continuing as though startup succeeded.



The executable locates its adjacent web bundle and loads it through the local WebView2 host.

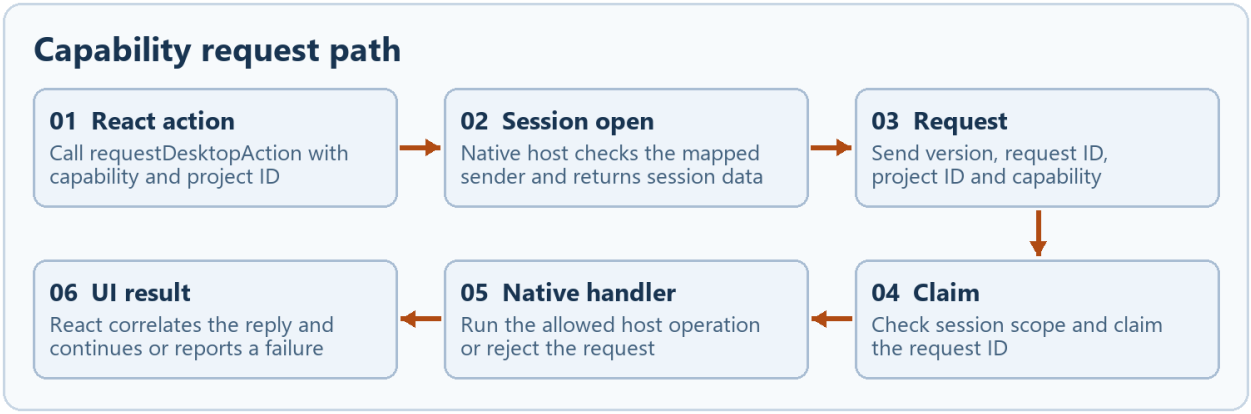
In React, the local origin uses HashRouter so client routes do not require native filesystem endpoints. Standalone browser use selects BrowserRouter. To investigate a blank window, begin with the native status and Diagnostics, then verify index.html, its referenced hashed assets, the loader architecture and the installed WebView2 Runtime. A working browser development server alone does not validate the desktop stage.

Source map: desktop/DelphiWebStudio/DelphiWebStudioPro.dpr, WebStudio.Main.dfm, WebStudio.Main.pas and src/App.tsx.

The native bridge grants narrow capabilities

The editor uses window.chrome.webview on the mapped desktop origin to request native actions. requestDesktopAction opens a versioned session for a named capability such as project.open, project.save, media.import or site.export, then sends a request bound to that capability and the current project. Each exchange has its own request identity and matching response, allowing the caller to distinguish authorization from later completion.

On the native side, the WebMessage handler checks the sender, message size and JSON fields before dispatch. WebStudio.Bridge.pas binds the grant to the project and capability and rejects replayed requests. Navigation revokes the current session, preventing a later document from simply inheriting the old grant. Separate event handlers govern top-level navigation, frames, new windows and permission requests.



The capability exchange authorizes an operation. The later operation still has to succeed in its own subsystem.

Font and color dialogs, credential requests, dirty-state reports and save-completion messages have their own validated handlers. Extend the specific contract that owns the operation rather than adding a generic native command facility. When debugging, record non-sensitive request kinds and IDs, and separate a rejected session from a failed file dialog or filesystem write.

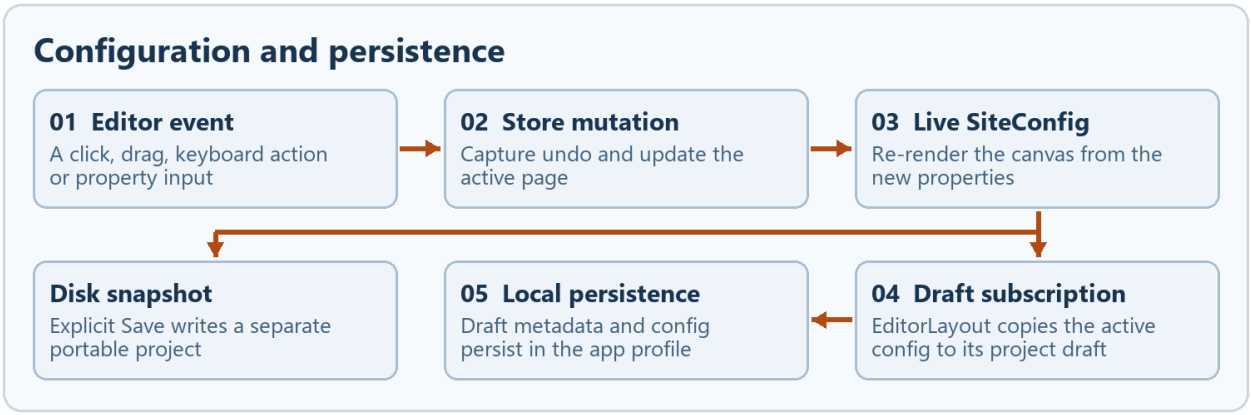
Source map: src/desktop/native-bridge.ts, src/desktop/bridge-protocol.ts, desktop/bridge/WebStudio.Bridge.pas and WebStudio.Main.pas.

State ownership explains what can change independently

The live configuration, local draft, media store and confirmed disk file are separate states. An edit updates the configuration first. EditorLayout subscribes to that configuration and updates the active draft, while media bytes live in their own IndexedDB namespace. A portable save captures a snapshot and reaches disk only after its native operation completes.

Area	Implementation
useConfigStore	The live SiteConfig, active page and undo/redo snapshots. Editing actions update the active page and preserve the Home compatibility block array.

Area	Implementation
useProjectsStore	Local draft catalog, project metadata, portable-file extension fields, per-project settings and media revision. Persisted in the WebView or browser storage.
useEditorStore	Session UI state, including active project, selection, panel/placement modes, viewport, preview, busy state and native save status.
IndexedDB assets	Original media bytes and metadata are stored per project. The serializable configuration references them by ID or path.
IndexedDB revisions	Explicit snapshots are independent of the editing session and its undo stack.



Rendering and local draft updates can happen before a new version is confirmed on disk.

This separation is important when a save appears to succeed but the editor remains dirty. A newer edit may have arrived after the snapshot, or the native completion may not have matched its token. Equally, reopening a matching portable file can reuse a newer local draft instead of overwriting it. Trace the identities and snapshots involved before assuming that there is only one authoritative in-memory object.

Source map: src/store/configStore.ts, src/store/projectsStore.ts, src/store/editorStore.ts and src/editor/EditorLayout.tsx.

The portable project model preserves editable structure

SiteProjectFile wraps SiteConfig with a versioned format, project identity, settings and optional media metadata. The current envelope uses format webstudio and schemaVersion 2. A page owns an ordered list of blocks, each with a stable ID, type, variant and property map. Page-specific SEO, article and custom-code data sit beside the page content, while shared-section and visibility information can accompany blocks.

```
{
  "format": "webstudio",
  "schemaVersion": 2,
  "projectId": "engineering-example",
  "config": {
    "name": "Community Example",
    "blocks": [],
    "pages": [{ "id": "home", "name": "Home", "path": "/", "blocks": [] }]
  }
}
```

The top-level blocks view is maintained for compatibility, with normal store actions synchronizing it to Home. Validation checks paths, structure and supported data rules before a file is activated or exported. A modern multipage project requires its root page. Unknown extension information can survive a round trip, but preserving an unsupported block is different from having a renderer that can export it.

When adding a field, define how an older project without it behaves. Prefer an optional value with a documented default when that preserves compatibility, and test save, reopen, undo and export rather than only the new control. Source map: src/blocks/types.ts and src/project/project-file.ts.

Store actions connect edits history and shared sections

Configuration actions form the mutation boundary. mutateActivePageBlocks normalizes the page structure and maintains the compatibility blocks view. Tracked changes capture a structured configuration snapshot for undo, bound the history and clear the redo branch when a new edit begins. Opening a different configuration resets that session history; media replacement and native disk writes follow their own mechanisms.

Area	Implementation
Shared section	Sharing a Navbar or Footer creates or associates linked copies across pages. Stopping sharing removes the relationship while keeping copies for independent editing.
Reusable section	A project-owned template copy of a block. Inserting it creates an independent block and identity. The saved template can be reused again.
Changing a route	changePagePath validates the new address and can update supported link fields. Export also resolves paths relative to each output file. User-supplied custom code needs separate review.

updateBlockProps can propagate an edit to blocks that share a webstudioSharedSectionId. A reusable section instead supplies an independent copy. Test both relationships deliberately: editing one linked Navbar should update its linked peers, whereas changing an inserted reusable copy should not silently rewrite another copy.

For a route change, the store can walk known link fields through visitBlockLinks. The traversal covers Navbar children, image and section-object destinations, rich-text href values and supported Markdown links. Custom scripts that construct their own URLs remain outside that model. Source map: src/store/configStore.ts and src/project/page-links.ts.

Selection connects the canvas to focused properties

Canvas chooses the active page, resolves local media and applies theme variables to the viewport. BlockWrapper supplies the editing shell around a registered renderer, while BlockColumns provides the section containers. SectionObjects manages independent items and their identity as a block ID, section number and object ID. Older placed-media paths retain separate identities, which matters when supporting existing files.



Selection is an identity that connects the rendered item, its handles and the corresponding settings fields.

Layers, context menus and the selection toolbar should resolve that same identity. RightSidebar is the source module that now routes the floating settings window to an object, section, composite item or block. Its historical name does not imply that the interface should expose the former long sidebar. Single-click selection and double-click text entry must cooperate without replacing the intended panel selection.

Preview disables editing behavior. A recent regression showed why that boundary matters: selection work in pointer-down capture caused React to replace an anchor before mouse release, so a real click failed even though a scripted click passed. The capture path now exits for Preview. Test physical pointer events when changing this area, because element.click() does not exercise the full sequence.

Source map: src/editor/Canvas.tsx, RightSidebar.tsx, SelectionToolbar.tsx, src/blocks/BlockWrapper.tsx and SectionObjects.tsx.

Geometry panels and collages use different local spaces

Block positions and widths are relative to page width. Within a section, an object’s x and width are percentages of that section, while y and height use pixels. Layer numbers order items within their local stack. BlockColumns persists one to four section widths and retains hidden extra-section data when the visible count is reduced.

```
// In a section 600 pixels wide, this image begins at 60 pixels
// and spans 240 pixels horizontally.
{ "id": "park-image", "type": "image", "x": 10,
  "y": 24, "width": 40, "height": 135, "layer": 2 }
```

A move into a different section must recalculate percentage geometry from the destination bounds while preserving the pointer’s grab offset. The movement action also transfers attached children and rich-text properties. Snapping compares nearby edges and centers; visible smart guides report the relationship and an enabled snap option applies the adjustment.

Panels establish ownership through parentId. Mere overlap does not create that relationship, so moving a panel must distinguish attached contents from unrelated nearby objects. A collage can be a full block or an independent section object with its own image geometry. Conversion into a section is one undoable mutation and rejects source conditions that would drop shared or unrelated content.

Source map: BlockColumns.tsx, SectionObjects.tsx, BlockWrapper.tsx, collage/CollageBlock.tsx and the corresponding configStore actions.

Rich text combines editable DOM with persistent runs

RichText keeps editable content and formatting connected to a property in the block model. Browser selection supplies the range for font, color, emphasis and alignment operations. Rich runs preserve formatting alongside plain text, and the current run structure can also retain href values and semantic line information for headings, paragraphs and list items.

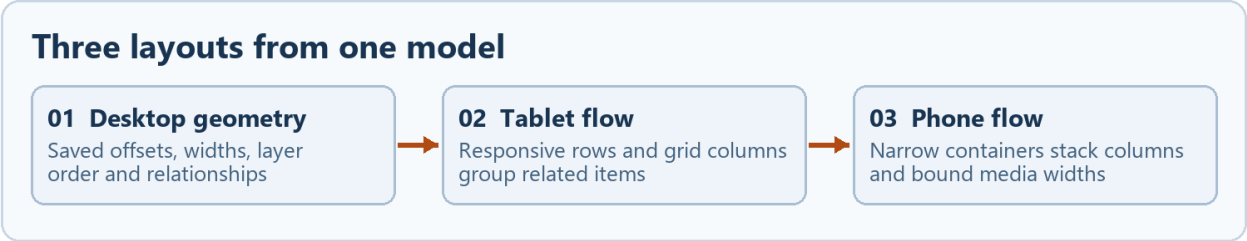
Section objects store formatting under names derived from the object ID, including sectionObject_<id>Rich. Any action that moves, duplicates or deletes a text object must account for those companion properties. Copying only its visible string would lose formatting and links even if the text still appeared in the destination.

Inline Link and Ctrl+K use the active selection to create or remove links, with supported destinations validated before storage. Markdown content and rich-run rendering share semantic block classes so a link edit does not flatten the surrounding paragraph or list structure. Save dispatches webstudio-flush-text and blurs a live contenteditable before capturing the snapshot.

Avoid replacing or remounting the editable DOM while a live selection is being used. Preserve the range or deliberately finish the edit first, and test repeated editing, moving and saving of the same formatted object. Source map: src/blocks/RichText.tsx, src/lib/markdown.ts and configStore.ts.

Responsive layout and page navigation need parity

The desktop composition may use positioned objects, but narrower containers need a useful reading flow. Responsive grouping derives rows, columns and width hints from the saved arrangement, allowing related content to remain together while images stay bounded and groups stack when needed. The main content reflow rules in src/index.css and src/lib/export.css use 940 and 700 pixel container breakpoints; other component and visibility rules have their own thresholds.



The saved geometry informs responsive grouping, while editor and export styles must express the same intended layout.

Check the canvas width rather than only the outer application window. Desktop, Tablet and Mobile presets simulate that content container, and object handles must still reflect the visible item after it reflows. Page Base uses theme bg0, while blocks and panels may override their own fills. A resized block needs its selected background through the added height, not just behind the original child content.

Preview navigation has a separate scroll responsibility. Canvas tracks page changes in a layout effect, resets its own scroll container for ordinary navigation and resolves pending anchors when present. Same-page plain links also reset the position. Test a link from a scrolled source, a dropdown destination and an anchor; wait for an intentional smooth scroll to finish before measuring the next action.

Source map: src/editor/Canvas.tsx, src/blocks/SectionObjects.tsx, src/index.css, src/lib/export.css and src/lib/export-html.ts.

Media identity is independent of folder organization

ProjectMediaLibrary stores original media and metadata in the webstudio-media IndexedDB database. Its asset records are scoped by project and asset ID and include the Blob, size, MIME information and SHA-256 digest. Matching imported content can reuse an entry, remain a separate copy or be skipped according to the selected policy.

Logical folders are metadata. The project records empty folder paths, and each asset can carry its membership without changing the stable assets path or content. Manage Media changes that organization; MediaLibraryDialog also serves the picker that returns an asset to an existing property or a placement command. Keeping identity stable is what allows folder moves without reconnecting page images and PDFs.

Area	Implementation
Rename	Changes media metadata without requiring a new identity. Undo renames restores recorded prior names.
Replace	Substitutes bytes for an existing asset and can preserve the previous version for undo. The project media revision changes so the canvas refreshes the rendered media.
Copy and reopen	Copies or restores assets into the destination project namespace so two drafts can hold separate media.

At render time, Canvas resolves persistent references to temporary object URLs and revokes them when the effect is replaced or unmounted. Those URLs belong to the session and must not be written as portable references. During save or export, the library verifies recorded sizes and digests before packaging original bytes. Test organization by moving a referenced image and PDF, reopening the ZIP and comparing both paths and hashes.

Source map: src/media/media-library.ts, MediaPanel.tsx, MediaLibraryDialog.tsx and src/editor/Canvas.tsx.

PDFs and visitor components keep local interaction state

PdfBlock uses PDF.js and its worker to render a page into a canvas. Document identity, page, zoom and measured size determine the current render. The expanded bulletin dialog separates the toolbar from the scrollable reading area, allowing a larger page to move horizontally and vertically without hiding the close control.

The weekly-bulletins DocumentLibrary variant presents up to five valid PDF entries, with the first treated as current. Selecting a file prepends it while avoiding another row for the same asset. Preview opens the resolved local document in its reader, while exported bulletin links point to the original PDF. An embedded PDF Viewer has a separate exporter path that packages viewer assets.

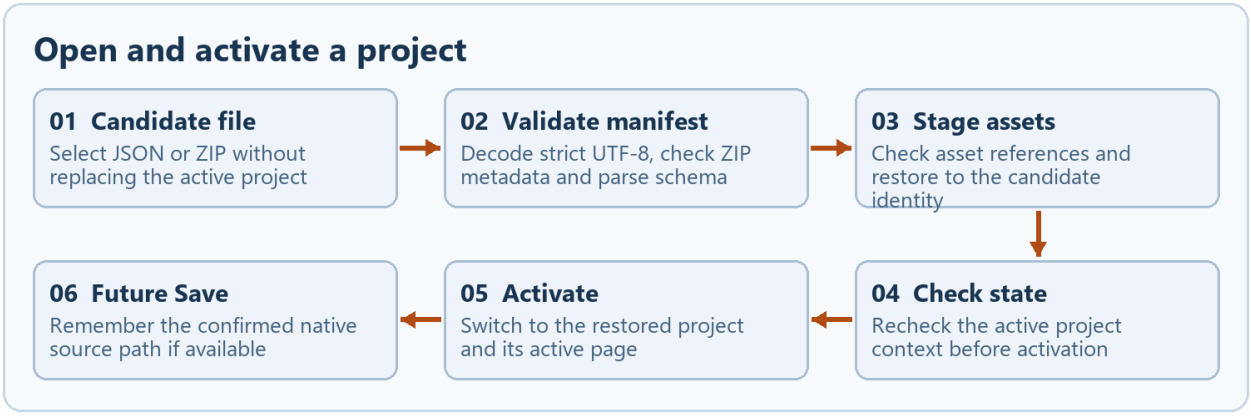
Other visitor components combine saved data with temporary UI state. Calendar stores events in block properties but changes the displayed month locally. Staff biographies measure whether the short preview overflows before offering a dialog. Galleries can use generated thumbnails while retaining originals for enlargement and captions. These interactions must not create authoring changes just because a visitor browses them.

Area	Implementation
Search	Opt-in site search generates its page and static search data from exported content. It is separate from editor-side Check site. Review the generated output after changing search indexing or exclusions.
Articles	Draft articles stay in the project. Ready articles export and can appear in the generated index. Author, summary and tags travel with the page metadata. A valid public base URL supports canonical links, sitemap and the article RSS feed.
Navigation	An ordinary page's links come from its configured Navbar or other content. The generated search and article index can reuse the Home Navbar. Sharing an editable Navbar is separate from this export behavior.
Contact	The project webmaster email supplies the default recipient when a Contact block has none. Preview shows behavior without transmitting. Export can use a valid configured HTTPS endpoint or open the visitor's email application.
Anchors	Names become element IDs when valid. Link handling distinguishes external URLs, page routes, same-page hashes and media.

The Navbar emits a parent anchor and a separate popover trigger. Its shared keyboard behavior closes Escape and restores focus to the trigger. Examine the shared scripts as well as the React renderer and HTML output when changing keyboard behavior. Source map: src/blocks/pdf, media/DocumentLibraryBlock.tsx, navbar, calendar, team, gallery, src/media/viewer.html and export-html.ts.

Opening compares snapshots before activating a project

ProjectFileCommands parses supported JSON or finds project.webstudio.json inside a ZIP. The reader checks manifest limits, recorded CRC and strict UTF-8, then validates or migrates the project. Media paths, sizes and digests are checked while original bytes are restored. Activation is deferred until the candidate and its media are ready.



A candidate project is prepared and checked before it replaces the active editor configuration.

Before allocating another draft, the open path compares portable project identity, configuration and saved metadata against recorded projectFile snapshots. A matching saved version reuses the existing local identity and preserves its newer draft configuration. A different snapshot can receive another identity. The portable project ID and the local draft ID therefore need to be traced separately.

Staging cleanup matters when media restoration fails. The operation should remove its own staged entries and leave the current project active. It also checks for relevant changes to the active session while preparation is in progress. A useful regression changes an image's bytes in a synthetic ZIP without updating its digest, then verifies that opening is rejected without losing the working project.

Source map: src/project/ProjectFileCommands.tsx, project-file.ts and desktop/project/WebStudio.ProjectOpen.pas.

Saving confirms a snapshot before clearing dirty state

The save path first flushes inline edits, captures configuration and settings, validates required references and creates a JSON or ZIP snapshot. Media-bearing saves include the manifest and original asset bytes. Sensitive deployment settings are removed from portable output, and the Gemini credential belongs to the Windows account store rather than the project.



The saved token identifies the completed snapshot. The editor clears dirty state only when that snapshot still matches current work.

In the desktop build, a unique token follows the download to the native host. The host uses the known project path for Save or obtains a new destination for Save as, stages the bytes and commits the file. project.save.complete returns the token and actual path. The editor compares its saved signature and input version with the current session before returning confirmation.

This handshake handles the common case where typing continues during save. A successful write must not mark later work as clean. When Save was requested on exit, the host waits for matching clean confirmation before closing; cancelled, failed or superseded saves leave the editor open. Regressions should test both the window X and File > Exit after ordinary Save, after Save as and after a new edit.

Source map: ProjectFileCommands.tsx, editorStore.ts, native-bridge.ts, WebStudio.Main.pas and WebStudio.ProjectSave.pas.

Native replacement and recovery protect different copies

The native save unit validates the staged project and checks the destination identity, size and write time before replacement. Windows ReplaceFile can temporarily retain the original beside the selected project. After checking the result and the old file identity, successful cleanup removes that temporary original so routine updates do not accumulate recovery directories.

If replacement or cleanup needs inspection, preserve the identified material and report its path. An interrupted download can be incomplete even when it resembles JSON or ZIP, so preliminary recognition is not proof of a valid project. Explicit backup and restore work from confirmed disk files, which can be older than unsaved editor state.

Area	Implementation
Explicit project backup	SaveProjectRecoveryZip writes a package from the current confirmed file. It works with disk state, which can be older than an unsaved draft.
Restore backup	RestoreProjectRecoveryZip writes and verifies the recovery contents before completing a restore. Open the result to verify the model and media.
Local revision	ProjectRevisions stores a separate Blob and checksum in IndexedDB. Opening creates another draft. A local revision is not the native backup ZIP.

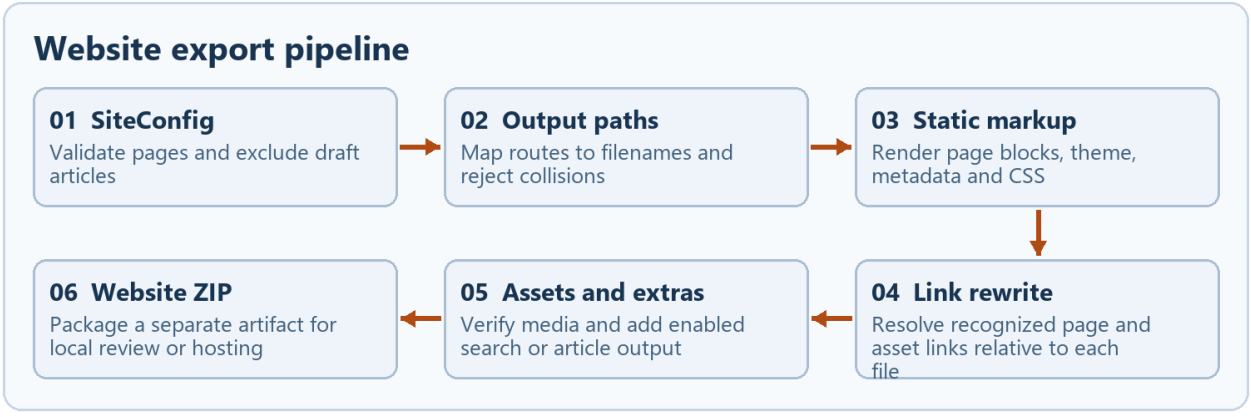
Local revisions use their own IndexedDB store, Blob and checksum. They offer useful checkpoints but remain dependent on the browser profile until exported as portable files. Tests should verify cancellation, a changed destination, a locked file, invalid staged content and successful reopening, without claiming complete power-loss durability from a controlled process test.

Source map: desktop/project/WebStudio.ProjectSave.pas, WebStudio.ProjectOpen.pas, WebStudio.Main.pas and src/project/project-revisions.ts.

Import and export transform the model at different boundaries

Static HTML import reads a bounded selection into a reviewable conversion and stages supported media before creating another project. WordPress import handles supported WXR content with its own limits and report. Neither converter reconstructs arbitrary scripts, plugins or server behavior. Retained source is reference material and can contain content that should not be distributed with a public project.

The exporter renders supported blocks into page files, calculates relative links for their depth and includes referenced assets. Root / becomes index.html, while nested destinations are mapped to their generated files with query strings and fragments preserved. Draft article pages remain in project saves but are filtered from website output, search and sitemap generation.



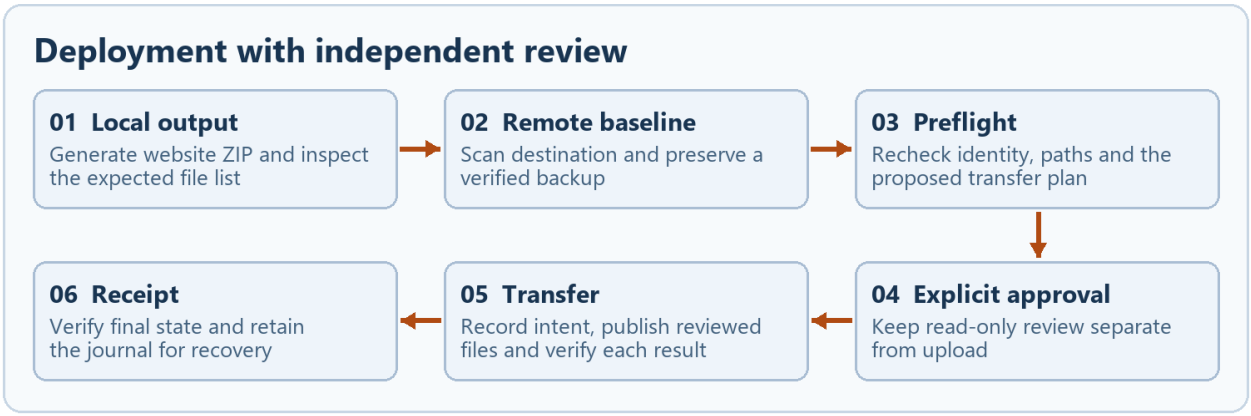
Website output is generated from the project model and has its own rendering and URL rules.

Generated search and article pages can borrow navigation from the root page. A maintenance change that promotes a different page to / therefore affects more than the landing filename: it can also affect borrowed navigation, legacy blocks synchronization and incoming Home links. Verify those relationships explicitly rather than only swapping two page names.

Source map: src/project/static-html-import.ts, wordpress-import.ts, src/lib/export-html.ts, export.css and website-manifest.ts.

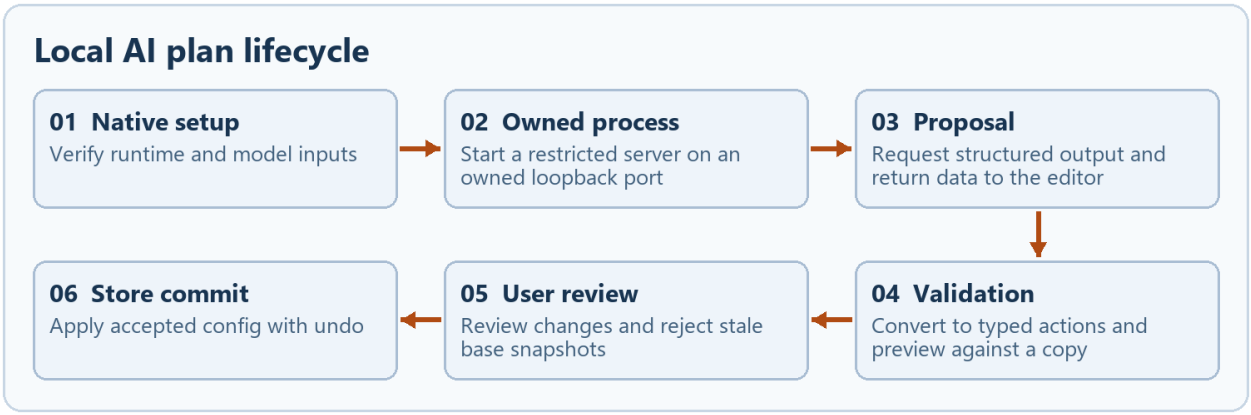
Deployment and AI remain explicitly reviewed operations

Website review prepares a file list and comparison material before any transfer. SFTP adds connection settings, destination inventory, backup and preflight. The executor records intent and completed operations in a journal and distinguishes additions from replacements. Rollback uses the receipt and recovery material rather than assuming an interrupted upload can simply be repeated.



Deployment review and journaled execution are separate from producing local website files.

Local AI also has a proposal boundary. The host validates supported runtime and model inputs, starts the restricted llama.cpp process on loopback, checks readiness and performs inference off the UI thread. React converts the returned data into supported actions, validates a proposed result and presents it for approval. Acceptance compares the current model with the reviewed base, rejecting a stale proposal.



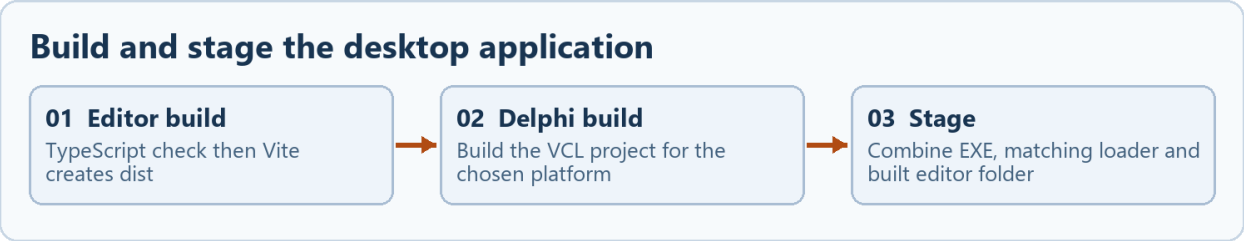
The model supplies a proposal. The reviewed and validated store action applies it to the project.

Cloud generation is another optional route. Windows Credential Manager can retain the Gemini key for the current account, while session consent controls use. Preserve origin validation, redact secrets from diagnostics and keep local inference distinct from direct remote or hosted API calls when testing. Source

map: desktop/project deployment units, desktop/ai, src/ai/site-actions.ts, LocalPlanReview.tsx, Settings.tsx and generate-site.ts.

Build a coherent package and test the affected boundary

Vite compiles the web editor after TypeScript checking. The resulting dist becomes editor beside each desktop executable. RAD Studio builds the native host from DelphiWebStudioPro.dproj for Win32 and Win64, with generated units under dcu and final stages under bin\Win32 and bin\Win64. Each stage needs its matching WebView2 loader.



Both architectures need a complete stage with matching native files and the intended editor bundle.

A web-only change requires a rebuilt and restaged editor bundle even when the native executable bytes do not change. A Pascal, DFM or resource change requires the corresponding native build. Verify the index references and file hashes in both stages rather than relying on the executable timestamp to identify the editor revision.

Area	Implementation
TypeScript checks	Run npm run lint, npm test and npm run build. These exercise source and bundle checks. GitHub's checks workflow is manually triggered in the inspected revision.
Project and media tests	project-file.test.ts, project-storage.test.ts, project-revisions.test.ts and media-library.test.ts cover model, storage and archive behaviors.
Editor tests	editor-selection.test.ts, floating-settings.test.ts, selected-object-properties.test.ts, navbar-properties.test.ts and block-columns.test.ts focus on state and markup contracts.
Export and AI tests	export-html.test.ts, multipage-export.test.ts, website-manifest.test.ts and site-actions.test.ts exercise output, path rules and proposal acceptance.
Browser test	tests/browser/professional-ui.mjs targets the editor UI. Use isolated profiles and a fixture so edits and browser storage are disposable.
Native tests	tests/native contains Delphi harnesses for bridge, save, recovery, export and AI. SFTP scripts support controlled loopback testing. Run native coverage against the relevant loader and Win32 or Win64 target.

Area	Implementation
Blank app window	Start at FormCreate, the native startup status and Diagnostics. Confirm the mapped editor directory, bundle hash names and matching runtime/loader.

Area	Implementation
Cannot select or move	Inspect the hit target, active editing state, block ID, section and object ID. Compare Canvas capture, SectionObjects and Layers before changing store logic.
Change appears in editor only	Find the mutation and saved properties. Trace the corresponding static renderer and CSS. Check whether a legacy placement path or custom code owns the final output.
ZIP opens without media	Verify manifest paths, asset sizes and digests, then inspect the project's IndexedDB namespace. Check for persistent references that accidentally used object URLs.
Save remains dirty	Trace webstudio-flush-text, the saved signature, input version and token. Compare project.save.complete and project.save.confirmed. Inspect the native save error before touching user files.
Wrong file on the host	Compare the local export, reviewed manifest, destination scan and deployment receipt. Recheck the destination rather than rerunning an interrupted upload blindly.

Choose evidence that fits the change: disk bytes and reopening for a save fix, matched Editor and exported layouts for responsive work, and physical pointer events for click behavior. Record the architecture, revision and remaining limits; clean-Windows and hosting acceptance need separate evidence. Keep tests in isolated profiles and remove their staging, extracted sites and process instances when finished.

Area	Implementation
Startup and forms	desktop/DelphiWebStudio/DelphiWebStudioPro.dpr · WebStudio.Main.dfm · WebStudio.Main.pas
Native message contract	src/desktop/native-bridge.ts · src/desktop/bridge-protocol.ts · desktop/bridge/WebStudio.Bridge.pas
Page model and undo	src/blocks/types.ts · src/store/configStore.ts · src/store/projectsStore.ts · src/store/editorStore.ts
Canvas and layout	src/editor/Canvas.tsx · src/blocks/BlockWrapper.tsx · src/blocks/BlockColumns.tsx · src/blocks/SectionObjects.tsx · src/index.css
Project and assets	src/project/ProjectFileCommands.tsx · src/project/project-file.ts · src/project/project-revisions.ts · src/media/media-library.ts
Native storage	desktop/project/WebStudio.ProjectOpen.pas · WebStudio.ProjectSave.pas
Export and SFTP	src/lib/export-html.ts · src/lib/export.css · src/lib/website-manifest.ts · desktop/project/WebStudio.SftpInventory.pas · WebStudio.DeploymentExecutor.pas
Local AI and proposals	desktop/ai/WebStudio.LocalAIHost.pas · WebStudio.LocalAIProcess.pas · WebStudio.LocalAIClient.pas · src/ai/site-actions.ts
Packaging and checks	desktop/installer/DelphiWebStudioPro.iss · .github/workflows · package.json · tests/native